INSTITUTO FEDERAL CATARINENSE - IFC
BACHARELADO DE CIÊNCIA DA COMPUTAÇÃO

ANDRÉ VITOR BASTOS DE MACÊDO

ALGORITMOS DE BUSCA DE CAMINHO EM REPRESENTAÇÕES NAVEGÁVEIS TRIDIMENSIONAIS : Uma análise estatística e comparativa

Blumenau
2026

ANDRÉ VITOR BASTOS DE MACÊDO

ALGORITMOS DE BUSCA DE CAMINHO EM REPRESENTAÇÕES NAVEGÁVEIS TRIDIMENSIONAIS : Uma análise estatística e comparativa

Trabalho de Conclusão de Curso apresentado ao curso de Bacharelado em Ciência da Computação do Instituto Federal Catarinense.

Orientador(a): Prof. Paulo César Rodacki Gomes

Blumenau
2026

LISTA DE ILUSTRAÇÕES

Figura 1 Comparação entre representações 2D e 3D 7
Figura 2 Exemplo de grafo simples. 9
Figura 3 Exemplo de grafo direcionado e não direcionado. 10
Figura 4 Exemplo de grafo ponderado e não ponderado. 11
Figura 5 Exemplo de caminho em um grafo. 12
Figura 6 Exemplo de componentes conexos em um grafo. 12
Figura 7 Diversos caminhos possíveis entre dois vértices em um grafo. 12

LISTA DE TABELAS

LISTA DE ABREVIATURAS E SIGLAS

SIG Sigla

SUMÁRIO

1 INTRODUÇÃO 7
1.1 Contribuições 8
1.2 Objetivos 8
2 FUNDAMENTAÇÃO TEÓRICA 9
2.1 Grafos 9
2.1.1 Definição 9
2.1.2 Direcionamento 10
2.1.3 Pesos 10
2.1.4 Caminho 11
2.1.5 Componente conexo 12
2.2 Busca 12
2.2.1 Dijkstra 13
2.2.2 Heurísticas 13
2.2.3 A* 13
2.2.4 Weighted A* 13
2.2.5 Greedy Best-First Search 13
2.3 Geração Procedural de Terrenos 13
2.3.1 Heightmap 14
2.3.2 Perlin 14
2.4 Representações Geométricas de Terrenos 14
2.4.1 Voxels 14
2.4.2 Marching Cubes/Transvoxel 14
2.5 Representações Navegáveis de Terrenos 14
2.5.1 Grade 2.5D 14
2.5.2 Superfície Voxel 14
2.5.3 Grafo de Vértices 15
2.5.4 Grafo Dual 15
2.6 Avaliação de desempenho e estatísticas 15
2.7 Trabalhos relacionados 15
2.7.1 Johansson 15
2.7.2 Kapi 15
2.7.3 Matriz de diferença 15
3 DESENVOLVIMENTO 16
3.1 Pipeline 16
3.2 IFCG e visualizador concorrente 16
3.3 lwGraph<T> 16
3.4 search.hpp 16
3.5 Gerador de terrenos 16
3.6 GraphModel, GraphFactory, isTranversable 16
3.7 Instrumentação 16

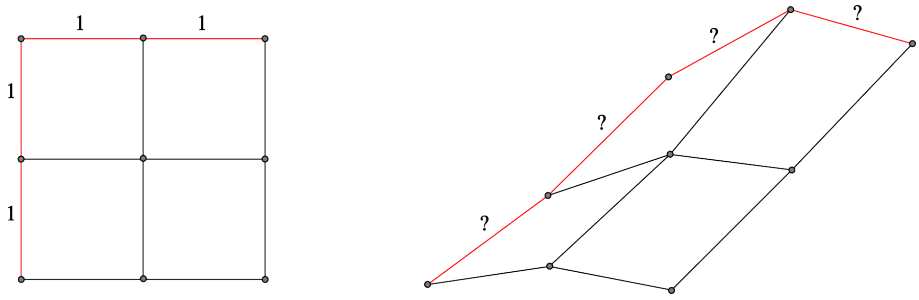
3.7.1 Statistics 16
3.7.2 acquire.sh 17
3.7.3 CLI 17
4 METODOLOGIA 18
4.1 Tipo de pesquisa 18
4.2 Ambiente 18
4.3 Variáveis 18
4.4 Níveis 18
4.5 Procedimento e métricas 18
4.6 Protocolo estatístico 18
4.7 Ameaças 18
5 RESULTADOS E DISCUSSÃO 19
6 CONCLUSÃO 20
REFERÊNCIAS 21

1 INTRODUÇÃO

A busca de caminhos (ou *pathfinding*) é um dos campos mais tradicionais da Inteligência Artificial aplicada, evoluindo a partir dos algoritmos de busca clássicos em grafos. Como definem Russell; Norvig (2013), os métodos clássicos de busca resolvem problemas de estados, ações e custos, visando encontrar caminhos entre um ponto de partida e um objetivo por meio da exploração do espaço de busca. Ao longo das últimas décadas, essa fundamentação deixou de ser puramente abstrata e passou a ser aplicada diretamente na computação interativa. Conforme apontam Pardede et al. (2022), os algoritmos de busca tornaram-se componentes cruciais para o desenvolvimento de jogos modernos e simulações em tempo real.

No entanto, a transição do espaço de busca de 2D para 3D introduziu um aumento significativo na complexidade, exigindo adaptações às técnicas de busca e de representação do terreno. A escolha da representação do espaço transitável e do algoritmo de busca a ser utilizado tornou-se uma decisão crítica, impactando diretamente o desempenho e a qualidade dos caminhos encontrados.

Figura 1 – Comparação entre representações 2D e 3D



Fonte: Elaborado pelo autor.

Em estudos recentes, é comumente evitada a comparação de múltiplas representações navegáveis de terrenos 3D, com foco em apenas uma abornagem na maiora das vezes. Além disso, a análise estatística do desempenho dos algoritmos de busca em diferentes representações é frequentemente negligenciada, limitando a compreensão das interações entre a estrutura do terreno e o comportamento dos algoritmos.

Esse trabalho propõe uma análise comparativa e estatística do desempenho de algoritmos clássicos de busca de caminhos em diferentes representações navegáveis de terrenos 3D. A pesquisa busca responder à seguinte questão central: Qual é a influência da **representação navegável** de um terreno 3D e das **características do terreno** no desempenho e na qualidade da busca de algoritmos clássicos de busca?

1.1 Contribuições

Para responder à questão central, é necessário realizar uma análise detalhada das representações navegáveis de terrenos 3D, dos algoritmos de busca aplicados e das métricas de desempenho utilizadas. Com isso, as principais contribuições deste trabalho incluem:

1. **Comparação entre representações navegáveis.**
   As mesmas instâncias de terreno são convertidas em quatro grafos diferentes e avaliadas com os mesmos algoritmos:
   - a partir do mapa de altura: grade de altura (2.5D) e superfície voxel;
   - a partir da malha poligonal: grafo de vértices e grafo dual.
2. **Metodologia experimental reprodutível.**
   Os terrenos são gerados proceduralmente a partir de parâmetros controlados (escala, lacunaridade, persistência) e semente registrada, e os resultados passam por análise estatística (ANOVA e testes post hoc).
3. **Implementação open-source.**
   Inclui uma biblioteca de grafos leve e o motor gráfico IFCG, com visualização das malhas e dos caminhos.

1.2 Objetivos

O objetivo geral do estudo é analisar comparativa e estatisticamente o desempenho de algoritmos clássicos de *pathfinding* em diferentes representações navegáveis de terrenos 3D. Mais especificamente, os objetivos incluem:

1. Implementar uma biblioteca de grafos e os algoritmos Dijkstra, A*, Weighted A* e Greedy Best-First.
2. Desenvolver um gerador procedural de terrenos baseado em ruído de Perlin, parametrizado por escala, lacunaridade e persistência.
3. Construir quatro representações navegáveis a partir do terreno: grade de altura, superfície voxel, grafo de vértices e grafo dual da malha.
4. Desenvolver um ambiente 3D com execução concorrente para visualização das malhas e dos caminhos encontrados.
5. Automatizar a coleta de métricas e analisar estatisticamente os resultados obtidos.

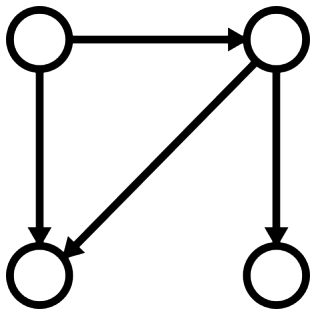
2 FUNDAMENTAÇÃO TEÓRICA

A comparação proposta neste trabalho depende de três conjuntos de conceitos: a teoria de grafos, que modela o espaço de busca; os algoritmos que percorrem esse modelo; e os métodos de representação de terrenos que determinam como o grafo é construído a partir da geometria. Este capítulo apresenta cada um deles, nessa ordem, e termina com os trabalhos que compararam representações semelhantes.

2.1 Grafos

Grafos são estruturas matemáticas que representam relações entre objetos. Eles consistem em um conjunto de vértices (ou nós) e um conjunto de arestas (ou ligações) que conectam os vértices. A representação de grafos pode ser direcionada ou não direcionada, ponderada ou não ponderada, dependendo das características do problema a ser modelado.

Figura 2 – Exemplo de grafo simples.



Fonte: Elaborado pelo autor.

Quando trabalhamos com estruturas de dados complexas, como terrenos 3D, os grafos fornecem uma representação abstrata que facilita a aplicação de algoritmos de busca. Cada vértice representa uma posição no ambiente, enquanto as arestas representam a transitabilidade entre esses pontos.

2.1.1 Definição

Segundo Gomes (2022), um grafo G é definido como um par ordenado (Equação 1), onde: G é o grafo; V é o conjunto de vértices; e E é o conjunto de arestas. Cada aresta representa a coneção entre dois vértices, podendo ter uma direção associada e um peso.

$$G = (V, E) \tag{1}$$

Cada conjunto de vértices e arestas pode ser representado de diferentes maneiras, dependendo da aplicação e das operações que serão realizadas sobre o grafo. As representações mais comuns incluem listas de adjacência, matrizes de adjacência e listas de arestas (Gomes, 2022).

Dado que um grafo é uma estrutura de dados baseada em conjuntos, naturalmente existem subgrafos. Subgrafos são subconjuntos de vértices e arestas de um grafo, que podem ser utilizados para analisar partes específicas da estrutura ou para simplificar a resolução de problemas complexos.

2.1.2 Direcionamento

Na criação de um grafo, é possível defini-lo como direcionado (Figura 3.a) ou não direcionado (Figura 3.b). Isso é, a conexão entre dois vértices pode ser unidirecional ou bidirecional.

Figura 3 – Exemplo de grafo direcionado e não direcionado.



(a) Grafo direcionado (b) Grafo não direcionado

Fonte: Elaborado pelo autor.

Como visto na Figura 3, em um grafo direcionado, as arestas possuem uma direção específica, indicando o sentido da conexão entre os vértices. Já em um grafo não direcionado, as arestas não possuem direção, permitindo a travessia em ambos os sentidos.

A escolha entre grafos direcionados e não direcionados depende do problema a ser modelado. Em muitos casos, como em mapas de ruas ou redes de transporte, as conexões são bidirecionais, enquanto em outros, como fluxos de dados ou hierarquias, as conexões podem ser unidirecionais.

2.1.3 Pesos

Grafos também podem ser ponderados (Figura 4.a), atribuindo valores numéricos às arestas para representar custos ou distâncias entre os vértices.

Figura 4 – Exemplo de grafo ponderado e não ponderado.



(a) Grafo ponderado (b) Grafo não ponderado

Fonte: Elaborado pelo autor.

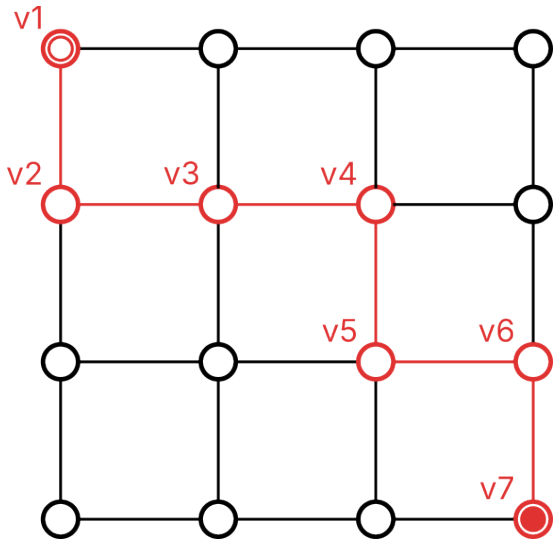
Ponderar a travessia entre vértices permite representar de forma mais precisa as características do ambiente, como a dificuldade de atravessar determinados terrenos ou a distância entre pontos. Como visto na Figura 4, a atribuição de pesos às arestas pode influenciar diretamente a escolha do caminho mais eficiente ou mais curto.

O custo de travessia de uma aresta pode representar diferentes aspectos, como distância física, tempo de viagem ou dificuldade de navegação. A escolha de atribuir pesos às arestas depende do contexto do problema e dos objetivos da estrutura.

2.1.4 Caminho

Um caminho em um grafo é uma sequência de vértices conectados por arestas, representando uma rota entre dois pontos (Gomes, 2022), ilustrado na Figura 5. A definição formal de um caminho é uma sequência de vértices $(v_1, v_2, ..., v_n)$ tal que cada par consecutivo $\{v_i, v_{i+1}\}$ está conectado por uma aresta no grafo.

Figura 5 – Exemplo de caminho em um grafo.



Fonte: Elaborado pelo autor.

Considerando v_1 o vértice inicial e v_7 o vértice final, a sequência de vértices $(v_1, v_2, v_3, v_4, v_5, v_6, v_7)$ representada na Figura 5 é o caminho $v_1 \rightarrow v_7$. Esse caminho é uma das várias rotas possíveis entre os vértices v_1 e v_7, e a escolha do caminho mais eficiente depende dos pesos das arestas e das características do grafo.

2.1.5 Componente conexo

Dado um grafo, um componente conexo é um subgrafo em que todos os vértices estão conectados entre si, permitindo a existência de caminhos entre quaisquer pares de vértices (Gomes, 2022).

Figura 6 – Exemplo de componentes conexos em um grafo.

Fonte: Elaborado pelo autor.

A identificação de componentes conexos é fundamental para entender a estrutura do grafo e determinar a viabilidade de encontrar caminhos entre diferentes regiões. Uma vez que um vértice pertence a um componente conexo, é possível garantir que existe um caminho para qualquer outro vértice dentro do mesmo componente.

2.2 Busca

A busca de caminhos em um grafo é o processo de encontrar uma sequência de vértices conectados por arestas que leva de um vértice inicial a um vértice objetivo. Diferentes algoritmos de busca podem ser aplicados, dependendo das características do grafo e dos requisitos do problema.

Figura 7 – Diversos caminhos possíveis entre dois vértices em um grafo.

Fonte: Elaborado pelo autor.

Um caminho encontrado não significa necessariamente que seja o mais curto ou o mais eficiente, uma vez que, a partir de um vértice inicial, podem existir múltiplos caminhos de pesos diferentes para um vértice objetivo. Por isso, algoritmos de busca são projetados para explorar o grafo de maneiras específicas, visando otimizar a descoberta de caminhos com base em critérios como custo, distância ou heurísticas.

2.2.1 Dijkstra

Algoritmo de busca de caminho mínimo em grafos ponderados, garantindo a descoberta do caminho mais curto entre um vértice inicial e todos os outros vértices.

2.2.2 Heurísticas

Funções que estimam o custo de um caminho entre dois vértices, ajudando a direcionar a busca para áreas mais promissoras.

fórmula: heuristica euclidiana = sqrt((x2 - x1)^2 + (y2 - y1)^2 + (z2 - z1)^2), onde (x1, y1, z1) e (x2, y2, z2) são as coordenadas dos vértices.

2.2.3 A*

Algoritmo de busca baseado no Dijkstra que utiliza heurísticas para encontrar o caminho mais curto entre dois vértices, combinando a precisão do seu pai com o uso de heurísticas.

fórmula: f(n) = g(n) + h(n), onde f(n) é o custo total estimado, g(n) é o custo do caminho até o vértice n e h(n) é a heurística estimada do vértice n até o objetivo.

2.2.4 Weighted A*

Algoritmo de busca que combina a abordagem do A* com um fator de ponderação, permitindo encontrar caminhos mais rápidos, mas possivelmente subótimos.

fórmula: f(n) = g(n) + w * h(n), onde w é o fator de ponderação que ajusta a influência da heurística na busca.

2.2.5 Greedy Best-First Search

Algoritmo de busca que utiliza apenas heurísticas para guiar a busca, priorizando a exploração de vértices mais promissores, mas sem garantir a descoberta do caminho mais curto.

fórmula: f(n) = h(n), onde f(n) é o custo total estimado e h(n) é a heurística estimada do vértice n até o objetivo.

2.3 Geração Procedural de Terrenos

Para gerar terrenos 3D de forma eficiente e realista, técnicas de geração procedural são empregadas, permitindo a criação de paisagens complexas e reprodutíveis a partir de algoritmos matemáticos.

2.3.1 Heightmap

Representação de terrenos 3D utilizando uma imagem em escala de cinza, onde a intensidade de cada pixel corresponde à altura do terreno naquele ponto.

figura: heightmap de elder scrolls, malha 3D gerada a partir do heightmap

2.3.2 Perlin

Algoritmo de geração de ruído contínuo, utilizado para criar padrões naturais e realistas em terrenos, texturas e outros elementos gráficos.

figura: ruidos de cada parâmetro, heightmap + malha final

2.4 Representações Geométricas de Terrenos

Formas de representar a geometria (visual) de terrenos em um ambiente 3D, utilizando diferentes técnicas e abstrações.

2.4.1 Voxels

Representação volumétrica de terrenos, onde o espaço é dividido em pequenas unidades cúbicas (voxels), permitindo a modelagem de formas complexas e detalhadas.

2.4.2 Marching Cubes/Transvoxel

Algoritmos de reconstrução de superfícies a partir de dados volumétricos, como voxels, permitindo a criação de malhas poligonais suaves e contínuas.

figura: exemplo de reconstrução de superfícies com Marching Cubes e Transvoxel

2.5 Representações Navegáveis de Terrenos

Formas de representar a navegação em terrenos, permitindo a movimentação e interação com o ambiente.

2.5.1 Grade 2.5D

Representação de terrenos em uma grade bidimensional com informações de altura. Baseada únicamente na altura do terreno.

figura: exemplo de grade 2.5D

2.5.2 Superfície Voxel

Representação de terrenos utilizando voxels, permitindo a modelagem de superfícies complexas e detalhadas, com informações de altura e volume.

figura: exemplo de superficie voxel

2.5.3 Grafo de Vértices

Representação de terrenos onde os vértices da malha do terreno representam pontos navegáveis.

figura: exemplo de grafo de vértices

2.5.4 Grafo Dual

Representação de terrenos onde os polígonos da malha do terreno representam regiões navegáveis, e as arestas conectam polígonos adjacentes.

figura: exemplo de grafo dual

2.6 Avaliação de desempenho e estatísticas

ANOVA e post-hoc de Tukey.

2.7 Trabalhos relacionados

2.7.1 Johansson

2.7.2 Kapi

2.7.3 Matriz de diferença

figura: matrix de diferença

3 DESENVOLVIMENTO

Neste capítulo, são apresentados os detalhes do desenvolvimento do trabalho, incluindo a arquitetura do sistema, a implementação dos algoritmos de busca, a geração de terrenos e a instrumentação para coleta de dados e testes.

3.1 Pipeline

Ruido -> Heightmap -> Representação Navegável -> Algoritmo de Busca -> Coleta de métricas -> Análise estatística

figura: pipeline do desenvolvimento

3.2 IFCG e visualizador concorrente

Motor gráfico para visualização de terrenos e caminhos encontrados pelos algoritmos de busca, permitindo a inspeção visual dos resultados.

figura: cena renderizada e diagrama taskmaster

3.3 IwGraph<T>

Estrutura de dados genérica para representar grafos, utilizada na implementação dos algoritmos de busca e na representação navegável dos terrenos.

3.4 search.hpp

Algoritmos de busca implementados em C++, incluindo Dijkstra, A*, WA* e GBFS, com suporte a aquisição de dados e coleta de métricas.

3.5 Gerador de terrenos

Ferramenta para geração de terrenos 3D, permitindo a criação de modelos realistas com diferentes características e propriedades.

figura: exemplo de diferentes representações navegaveis do mesmo terreno

3.6 GraphModel, GraphFactory, isTranversable

Classes para modelar, fabricar e verificar a traversabilidade de grafos.

3.7 Instrumentação

Ferramentas e técnicas para coleta de dados e testes, incluindo aquisição de métricas e análise estatística.

3.7.1 Statistics

Classe para coletar e exportar métricas.

3.7.2 acquire.sh

Script para aquisição de métricas, executando os algoritmos de busca em diferentes representações navegáveis de terrenos e coletando os resultados.

3.7.3 CLI

Interface de linha de comando para executar os testes, coletas ou exportações de dados.

4 METODOLOGIA

Neste capítulo, são descritos os métodos e procedimentos utilizados para conduzir a pesquisa, incluindo o ambiente experimental, as variáveis analisadas, os níveis de teste, o procedimento de coleta de dados e as métricas utilizadas para avaliação dos resultados.

4.1 Tipo de pesquisa

Definição do tipo de pesquisa realizada, incluindo a abordagem experimental, comparativa e estatística adotada para avaliar o desempenho dos algoritmos de busca em diferentes representações navegáveis de terrenos 3D.

4.2 Ambiente

Configuração do ambiente experimental, incluindo hardware, software, bibliotecas e ferramentas utilizadas para a implementação dos algoritmos de busca, geração de terrenos e coleta de métricas.

4.3 Variáveis

Variáveis independentes e dependentes analisadas na pesquisa, incluindo as representações navegáveis de terrenos, os algoritmos de busca e as métricas de desempenho coletadas.

figura: tabela de variáveis independentes e dependentes

4.4 Níveis

Definição dos níveis de teste para cada variável, incluindo diferentes representações navegáveis de terrenos, algoritmos de busca e configurações de parâmetros utilizados nos experimentos.

figura: tabela de níveis de teste para cada variável

4.5 Procedimento e métricas

Descrição do procedimento de coleta de dados e das métricas utilizadas para avaliar o desempenho dos algoritmos de busca.

4.6 Protocolo estatístico

Descrição do protocolo estatístico utilizado para análise dos dados coletados, incluindo testes de hipótese, intervalos de confiança e modelos de regressão.

4.7 Ameaças

Identificação e análise das ameaças à validade interna e externa do estudo, incluindo possíveis fontes de viés, limitações do método e impacto sobre os resultados obtidos.

5 RESULTADOS E DISCUSSÃO

6 CONCLUSÃO

REFERÊNCIAS

GOMES, Paulo César Rodacki. **Grafos: conceitos fundamentais, algoritmos e aplicações**. Blumenau, SC: Editora do Instituto Federal Catarinense, 2022.

PARDEDE, Sara Lutami *et al*. A review of pathfinding in game development. **Journal of Computer Engineering**, v. 1, n. 01, p. 47–56, 2022.

RUSSELL, Stuart; NORVIG, Peter. **Inteligência Artificial: uma abordagem moderna**. 3. ed. Rio de Janeiro: Elsevier, 2013.